Secure Application Integration Guide

1. Frontend: API Loading & Failure States

Implement logic to handle loading indications and failure statuses properly while fetching data from the API endpoint.

```
       import React, { useState, useEffect } from 'react';

       function ApplicationComponent() {
         const [data, setData] = useState(null);
         const [loading, setLoading] = useState(true);
         const [error, setError] = useState(null);

         useEffect(() => {
           fetch('https://your-api-endpoint.com/data')
             .then((res) => {
               if (!res.ok) throw new Error('Failed to fetch data from API');
               return res.json();
             })
             .then((data) => {
               setData(data);
               setLoading(false);
             })
             .catch((err) => {
               setError(err.message);
               setLoading(false);
             });
         }, []);

         if (loading) return Loading data, please wait...;
         if (error) return Error: {error};

         return Data Loaded Successfully!;
       }
```

2. Form Validation (Client & Server Side)

Always validate data fields on both the browser side (client) and backend logic blocks (server) to secure parameters comprehensively.

```
// Node.js/Express Server-Side Validation Example
app.post('/api/apply', (req, res) => {
    const { name, email } = req.body;

    if (!name || !email) {
        return res.status(400).json({ error: "All fields are required." });
    }
    if (!email.includes('@')) {
        return res.status(400).json({ error: "Please enter a valid email address." });
    }

    res.status(200).json({ message: "Form submitted successfully!" });
});
```

3. API Security Best Practices

Enforce rate limits, include rigid security headers via Helmet middleware, and prevent common vulnerabilities via Parameterized SQL queries.

```
// Secure Parameterized DB Queries
db.query('SELECT * FROM users WHERE email = ?', [userEmail], (err, results) => {
    // Process records safely without SQL Injection risks
});
```

Submission Instructions

Push your frontend and backend codes to a single **GitHub repository**. Add a README.md explaining local environment setups along with your custom verification criteria. Provide a public or view-only URL inside the portal's submit field.